



NTNU
Norwegian University of
Science and Technology

Binary fuzzing: Finding bugs without source code

Donn Morrison
Department of Computer Science

Introduction

Finding vulnerabilities

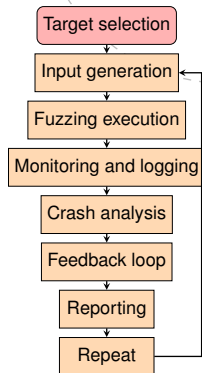
- Source code review
- Reverse engineering
- Fuzzing

Fuzzing

- What is fuzzing?
Mutating input data and repeatedly sending it to a program under test with the goal of eliciting a crash
- Why binary fuzzing?
We may lack the source code!

Fuzzing fundamentals

- Types of fuzzing: dumb, smart, coverage-guided
- Coverage: blackbox, whitebox, graybox
- Fuzzing targets: file parsers, network protocols
- Workflow: input generation, execution, monitoring, crash analysis



Dumb fuzzing

- Blindly randomly mutating input
- Advantages: simple, can be combined with coverage-guided
- Disadvantages: limited coverage (may not reach some code paths) due to structural agnosticism
- Example fuzzers: AFL/AFL++, honggfuzz, Angora

Smart fuzzing

- Uses knowledge of input structure to bypass parsers and stress corner cases
- Advantages: able to attain higher coverage, can be combined with coverage-guided
- Disadvantages: more difficult to create inputs
- Examples: network protocols, file formats (XML, JSON, etc)

Coverage-guided fuzzing

- Instruments the program to measure how input affects code path
- Advantages: measurable program coverage
- Disadvantages: adds overhead to program execution
- Examples: gray-box fuzzers like AFL/AFL++, honggfuzz, Angora

Binary fuzzing techniques

- Instrumentation: static vs. dynamic
 - Static: instrumented at compile time (afl-gcc)
 - Dynamic: instrumented at run time (afl-fuzz in QEMU mode)
- Harnessing binaries [2]
 - Consider coverage
 - Hook specific function calls
 - Modify how the binary reads input (e.g., using LD_PRELOAD)
 - Skeleton code - if fuzzing a library or have source code available, there's a lot you can do here
- Persistent mode - requires target modification
 - Target cannot have state
 - Much faster (executions/second)
 - Need start and end point addresses (AFL_QEMU_PERSISTENT_ADDR, AFL_QEMU_PERSISTENT_RET, and AFL_QEMU_PERSISTENT_GPR=1)
- Monitoring crashes

Binary fuzzing techniques cont.

Choosing your harness approach - consider fuzzing `httpd` as in [1].
Handles connections with `listen()/accept()`, reads data with `fgets()`.

- Blind via TCP/IP - easy, very slow, coverage ??
- Coverage-guided over TCP/IP - minimal target mod, slow
- Coverage-guided w/ desocketing - fast, desocketing difficult
- Coverage-guided persistent - very fast, target mod, no state, often impossible

QEMU mode (AFL++)

- No source code!
- Run afl-fuzz in QEMU mode (-Q), dynamic instrumentation
- afl-fuzz emulates target binary

Rehosting binaries (advanced)

- Binary from IoT/embedded device (e.g., MIPS), architecture different from where we want to fuzz (e.g., x86-64)
- IoT device resources lacking (RAM, CPU)
- Target often has specific hardware dependencies (e.g., IP camera) [3]

Fuzzing UI (AFL++)

```

american fuzzy lop ++4.04c {default} (./bo) [fast]

process timing
  run time : 0 days, 0 hrs, 0 min, 2 sec
  last new find : none seen yet
  last saved crash : 0 days, 0 hrs, 0 min, 2 sec
  last saved hang : none seen yet

cycle progress
  now processing : 0.2 (0.0%)
  runs timed out : 0 (0.00%)

stage progress
  now trying : havoc
  stage execs : 272/1175 (23.15%)
  total execs : 1713
  exec speed : 832.6/sec

fuzzing strategy yields
  bit flips : disabled (default, enable with -D)
  byte flips : disabled (default, enable with -D)
  arithmetics : disabled (default, enable with -D)
  known ints : disabled (default, enable with -D)
  dictionary : n/a
  havoc/splice : 1/1431, 0/0
  py/custom/rq : unused, unused, unused, unused
  trim/eff : 63.64%/2, disabled

overall results
  cycles done : 0
  corpus count : 1
  saved crashes : 1
  saved hangs : 0

map coverage
  map density : 0.00% / 0.00%
  count coverage : 1.00 bits/tuple

findings in depth
  favored items : 1 (100.00%)
  new edges on : 1 (100.00%)
  total crashes : 9 (1 saved)
  total tmounts : 0 (0 saved)

item geometry
  levels : 1
  pending : 1
  pend fav : 0
  own finds : 0
  imported : 0
  stability : 100.00%

[cpu000: 12%]
```

Demo fuzzing with AFL++

- Demo: fuzzing with source code (afl-gcc/afl-fuzz)
- Demo: fuzzing without source code (afl-fuzz -Q)
- More exercises [4]

Challenges and limitations

- Rehosting binaries
- No source code or documentation (reversing often required)
- Binary complexity: obfuscation, anti-debugging
- Performance: emulation overhead
- False positives/negatives

References

1. Fuzzing 101
<https://www.youtube.com/watch?v=SVoyjfMxfs0>
2. Rehosting Fuzzing Workshop Recon 2025
<https://github.com/srlabs/rehosting-fuzzing-workshop>
3. Harnessing libraries for effective fuzzing
<https://bushido-sec.com/index.php/2025/01/03/fuzzing-harness-guide/>
4. Fuzzing101 exercises
<https://github.com/antonio-morales/Fuzzing101>